

13

NETWORK AUTHENTICATION



The previous chapter discussed interactive authentication, which allows a user to log in to a computer and interact with a desktop. By contrast, *network authentication* occurs when the user has already authenticated to a Windows system but wants to use resources on another Windows system, typically over a network.

The simplest approach to performing network authentication might seem to be to transfer the user's credentials to the remote system. The service that receives the credentials could then call the `LsaLogonUser` API and specify the `Network` logon type to create a noninteractive logon session.

However, this approach isn't very secure. To use `LsaLogonUser`, a network-facing service must know the user's full credentials, and providing a remote system with these credentials is problematic for many reasons. For one, we must trust the remote service to handle the credentials securely. For another, if the authentication takes place over a hostile network, an attacker could capture the credentials.

To mitigate these security issues, Windows implements multiple network authentication protocols. These protocols don't require sending a network service the user's credentials or transferring a plaintext password over the network. (Of course, there are always caveats, which we'll identify over the course of this chapter.) You'll find these network authentication protocols in the security packages we discussed in the previous chap-

ter, and you can access them via a generic API, which allows an application to easily change the authentication protocol used.

This chapter begins by describing the *NT LAN Manager (NTLM)* authentication protocol, the oldest Windows protocol still in use, in some depth, covering how it uses the user's credentials to prevent their disclosure over the network. Then we'll look at a well-known attack, *NTLM relay*, and the ways Microsoft has tried to mitigate it.

NTLM Network Authentication

NTLM derives from the *LAN Manager (LM)* authentication protocol, which supported the Server Message Block (SMB) file sharing protocol as part of the LAN Manager operating system. Microsoft reimplemented the authentication protocol in Windows 3.11 (the infamous Windows for Workgroups), then built upon it further and dubbed it NTLM when it introduced Windows NT. In the latest versions of Windows, there are three variants of NTLM in use:

NTLMv1 The original NTLM version, introduced in Windows NT 3.1

NTLMv2 A version of NTLM introduced in NT 4 Service Pack 4 that added additional security features

NTLMv2 Session NTLMv1, but with the additional security features from NTLMv2

We'll focus on NTLMv2, the only version whose values are accepted by default on Windows Vista and above. You might still encounter NTLMv1 or NTLMv2 Session in mixed operating system environments (for example, when accessing Linux-based network storage devices), but in a modern Windows environment, these should be rare.

Figure 13-1 shows an overview of the NTLM authentication process that occurs between a Windows client application and a Windows server.

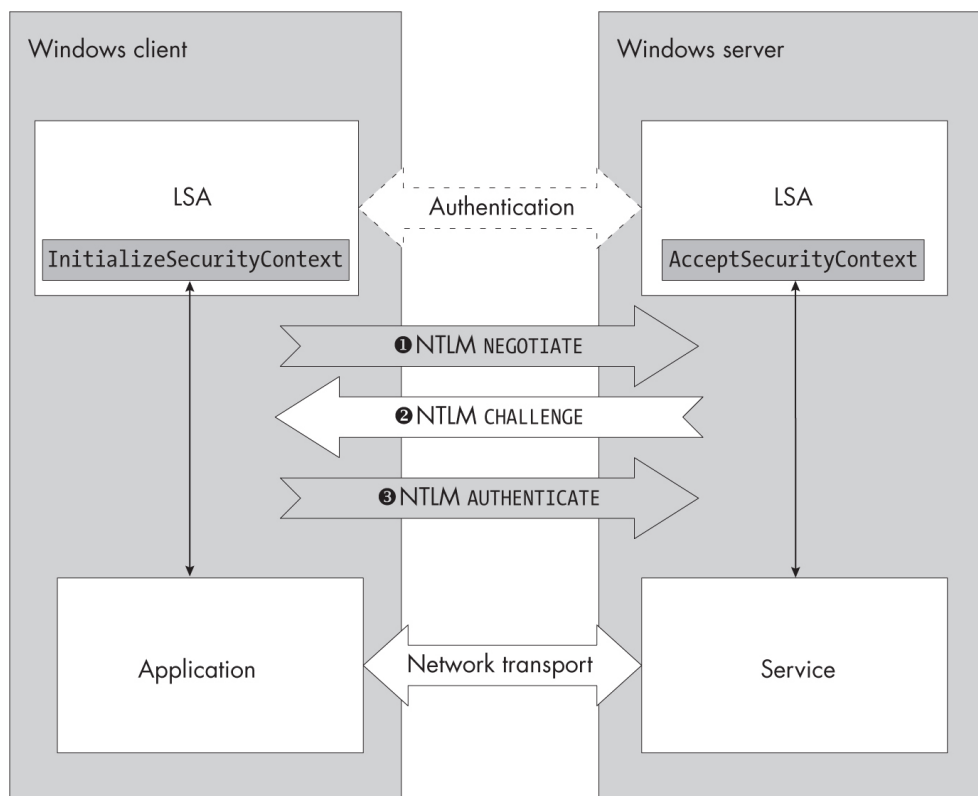


Figure 13-1: An overview of the NTLM authentication protocol

NTLM authentication begins when the client application makes a network connection to the server. The client and the server then exchange a sequence of binary *authentication tokens* generated by the LSAs on the two machines. For the NTLM, these tokens consist of three authentication messages: the client's `NEGOTIATE` message, which specifies which features the client supports ❶; the server's `CHALLENGE` message, which selects one of the client's features and provides a random challenge value to use in the exchange ❷; and the client's `AUTHENTICATE` message, which contains a value that proves the client's knowledge of the user's password to the server ❸.

At a high level, the authentication process occurs between the two LSAs. However, it's up to the application and server to transport these authentication tokens over some network protocol. Microsoft describes the authentication protocol in *MS-NLMP*, available online. The document omits some features, which I'll point out as we walk through an example.

NTLM Authentication Using PowerShell

Let's perform network authentication using PowerShell so you can see what information the authentication tokens contain. We'll use a local user account, though a domain account would work just as well.

Initializing the Client

The authentication process starts when the client application calls the Security Support Provider Interface (SSPI) API `AcquireCredentialsHandle`. The system implements the SSPI APIs to abstract the authentication protocol implemented by a security package. This allows applications to more easily change the network authentication protocol they use.

This `AcquireCredentialsHandle` API selects the security package used for network authentication and provides explicit credentials for the authentication if needed. It returns a handle for use by a second SSPI API, `InitializeSecurityContext`, which uses the selected security package but executes in the LSA.

The security package in the LSA processes `InitializeSecurityContext`, then requests and returns a `NEGOTIATE` authentication token to the caller. The `NEGOTIATE` token describes which authentication features the client supports and should be sent to the server over the network protocol. [Listing 13-1](#) performs this client initialization in PowerShell.

```
PS> $credout = New-LsaCredentialHandle -Package "NTLM"
           -UseFlag Outbound -UserName $env:USERNAME -Domain $env:USERDOMAIN
PS> $client = New-LsaClientContext -CredHandle $credout
PS> $negToken = $client.Token
PS> Format-LsaAuthToken -Token $negToken
<NTLM NEGOTIATE>
Flags: Unicode, Oem, RequestTarget, NTLM, AlwaysSign, ExtendedSessionSecurity,
Version, Key128Bit, Key56Bit
Version: 10.0.XXXXX.XX
```

Listing 13-1: Initializing a client for NTLM authentication and formatting a NEGOTIATE authentication token

We start by getting the credentials handle using the `New-LsaCredentialHandle` command, which calls `AcquireCredentialsHandle`. By specifying `NTLM` using the `Package` parameter, we select the use of the NTLM security package. We also specify that these credentials are for outbound authentication (that is, from a client to a server). Lastly, we specify the username and domain from the current environment.

Notice that we do not specify a password; this is because the LSA has already cached the password for us in our logon session. The fact that we don't need to specify the password is a key part of *Integrated Windows Authentication (IWA)*, which allows users to automatically authenticate to the network authentication using their credentials, without prompting them for a password.

With the credentials handle, we create a client authentication context by calling the `New-LsaClientContext` command and specifying the handle. Under the hood, this command calls the `InitializeSecurityContext` API. If the call to the API succeeds, the client context now contains the `NEGOTIATE` token. We store a copy of the token for later use, then pass it to the `Format-LsaAuthToken` command to parse its contents and print them to the shell.

The main component of the token is a list of flags that reflect the features the client requests, the features the client supports, and which parts of the token are valid. In this case, the token has nine flags set, though these can change depending on the system's configuration. **Table 13-1** shows what the flags mean in this context.

Table 13-1: Select NTLM Flags

Flag name	Description
Unicode	The client supports Unicode strings.
Oem	The client supports byte character strings (for example, ASCII).
RequestTarget	The client requires the server to send a target name in the response.
NTLM	The client requests to use the NTLM hash.
AlwaysSign	The client requests that the authentication be signed to ensure integrity.
ExtendedSessionSecurity	The client requests NTLMv2 Session security.

Flag name	Description
Version	The client has sent the operating system and NTLM protocol version.
Key128Bit	The client requests a 128-bit signing key.
Key56Bit	The client requests a 56-bit signing key.

You might wonder why the `ExtendedSessionSecurity` flag is set. This flag changes NTLMv1 to NTLMv2 Session security, but I mentioned earlier that NTLMv1 is disabled by default. The LSA sets the flag anyway, just in case the server responds with a request for NTLMv1. Except for `Version`, these flags all indicate the features the client requires. The `Version` flag indicates major, minor, and build values of the operating system version, as well as the NTLM protocol version, which has been fixed at 15 since Windows Server 2003.

To ensure the integrity of the authentication protocol, NTLM generates an encryption key based on the values in the exchange, then uses it to apply a *message integrity code (MIC)* to the entire exchange. A MIC is a cryptographic hash of the authentication tokens sent and received in the current exchange. It's used to detect whether the authentication tokens have been tampered with over the network.

Due to cryptography export restrictions, NTLM supports 40-bit keys as well as 56-bit and 128-bit keys, based on the presence of the `Key56Bit` and `Key128Bit` flags. If neither flag is set, NTLM will use 40-bit keys. The `Format-LsaAuthToken` command hides the underlying binary value of the authentication token, but to see the token in hex we can pass the `AsBytes` parameter to this command, as shown in [Listing 13-2](#).

```
PS> Format-LsaAuthToken -Token $client.Token -AsBytes
      00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F - 0123456789ABCDEF
-----
❶ 00000000: 4E 54 4C 4D 53 53 50 00 01 00 00 00 07 82 08 A2 - NTLMSSP.....
   00000010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 - .....
   00000020: 0A 00 BA 47 00 00 00 0F - ...G....
```

Listing 13-2: Formatting the authentication token in hex

In hex, we can see the data has a format indicator at the beginning of NTLMSSP ❶. If you see this indicator in data you're analyzing, there's a good chance you've come across an NTLM network authentication process. I won't display the hex of the rest of the tokens, as it's easy enough to change the script to view the hex output if you're interested in doing so.

Initializing the Server

The client has initialized its authentication context and generated a NEGOTIATE token. Now it must send this token to the server application so that it can initialize its own authentication context. When the server receives the token, it passes it to the LSA using the `AcceptSecurityContext` API. The LSA inspects the token, determines whether it supports the requested features, and generates a CHALLENGE authentication token in response. This token allows the server to verify that the client isn't replaying values captured from a previous authentication exchange.

Let's use PowerShell to demonstrate the server's handling of NTLM. In **Listing 13-3**, we create the server authentication context in the same process as the client's (remember, however, that the server will typically run on a different system).

```
PS> $credin = New-LsaCredentialHandle -Package "NTLM" -UseFlag Inbound
PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Token $client.Token
PS> $challengeToken = $server.Token
PS> Format-LsaAuthToken -Token $server.Token
<NTLM CHALLENGE>
Flags      : Unicode, RequestTarget, NTLM, AlwaysSign, TargetTypeDomain,
ExtendedSessionSecurity, TargetInfo, Version, Key128Bit, Key56Bit
TargetName: DOMAIN
Challenge  : D568EB90F6A283B8
Reserved   : 0000000000000000
Version    : 10.0.XXXXXX.XX
=> Target Info
NbDomainName - DOMAIN
NbComputerName - GRAPHITE
DnsDomainName - domain.local
DnsComputerName - GRAPHITE.domain.local
```

```
DnsTreeName - domain.local  
Timestamp - 5/1 4:21:17 PM
```

Listing 13-3: Initializing the server for NTLM authentication and formatting the CHALLENGE authentication token

We start by creating the inbound credentials handle. You don't need to provide any credentials to do this; in fact, NTLM would ignore the credentials even if you did provide them. Next, we create the server's authentication context and provide the client's `NEGOTIATE` token to the `AcceptSecurityContext` API by calling the `Update-LsaServerContext` PowerShell command. If the LSA accepts the `NEGOTIATE` token, the server context will include its own token, the `CHALLENGE` token. As before, we capture the token for later use and pass it to `Format-LsaAuthToken` to print out the information it contains.

The token's flags represent the values that the network authentication process supports and are based on the flags the client sent. For example, in [Listing 13-1](#) we saw that the client set both the `Oem` and `Unicode` string format flags in its `NEGOTIATE` token, indicating that it can support both Unicode and byte character format strings. The server has elected to send strings in Unicode format, so it has cleared the `Oem` flag in the `CHALLENGE` token.

As requested by the client, the output also contains the `TargetName`, which in this case is the domain name of the server, indicated by the `TargetTypeDomain` flag. If the server were not in a domain network, the `TargetName` would be the server's computer name, and the token would use the `TargetTypeServer` flag instead.

The `CHALLENGE` token contains a random 8-byte server challenge generated by the LSA. All values calculated in the next step depend on the challenge's value; because it's different for every request, this prevents an attacker from capturing a previous authentication exchange and replaying it to the server. The final part of the token is the target information, indicated by the presence of the `TargetInfo` flag. This contains additional details about the server.

Note that NTLM can work in a connectionless mode, in which the client never sends the initial `NEGOTIATE` message. In this case, the authentication

process starts with the CHALLENGE message from the server. However, connectionless NTLM authentication is rarely used in practice.

Passing the Token Back to the Client

Next, the server must send the CHALLENGE token to the client's authentication context. In a real network protocol, this would happen over the network, but in [Listing 13-4](#) we pass the token in the same script.

```
PS> Update-LsaClientContext -Client $client -Token $server.Token
PS> $authToken = $client.Token
PS> Format-LsaAuthToken -Token $client.Token
<NTLM AUTHENTICATE>
Flags      : Unicode, RequestTarget, NTLM, AlwaysSign, ExtendedSessionSecurity,
TargetInfo, Version, Key128Bit, Key56Bit
Domain     : GRAPHITE
UserName   : user
Workstation: GRAPHITE
LM Response: 0000000000000000000000000000000000000000000000000000000000000000 ❶
<NTLMv2 Challenge Response>
NT Response      : 532BB4804DD9C9DF418F8A18D67F5510 ❷
Challenge Verison : 1
Max Challenge Verison: 1
Reserved 1       : 0x0000
Reserved 2       : 0x00000000
Timestamp        : 5/1 5:14:01 PM
Client Challenge  : 0EC1FF45C43619A0 ❸
Reserved 3       : 0x00000000
NbDomainName - DOMAIN
NbComputerName - GRAPHITE
DnsDomainName - domain.local
DnsComputerName - GRAPHITE.domain.local
DnsTreeName - domain.local
Timestamp - 5/1 5:14:01 PM
Flags - MessageIntegrity ❹
SingleHost - Z4 0x0 - Custom Data: 0100000000200000 Machine ID: 5FB8... ❺
ChannelBinding - 00000000000000000000000000000000 ❻
TargetName -
</NTLMv2 Challenge Response>
MIC          : F0E95DBEB53C885C0619FB61C5AF5956 ❼
```

Listing 13-4: Updating the client for NTLM authentication and formatting the AUTHENTICATE token

We use the `Update-LsaClientContext` command, which calls `InitializeSecurityContext` once again with the original credentials handle and the `CHALLENGE` token. If `InitializeSecurityContext` accepts the token, the LSA generates the final `AUTHENTICATE` token, which we can then format. This is the only token that depends on the value of the password; the other two tokens can be generated without any special knowledge.

The `AUTHENTICATE` token starts with the final negotiated flags and the information about the user, including their username and domain. Because we're using a local account, the domain is set to the workstation name, `GRAPHITE`. Next comes the LM response, which in this case is all zeros ❶. The LM response is normally disabled, which is why it's not specified, and NTLMv2 doesn't use the LM hash at all.

We now continue to the full NTLMv2 response, which contains a lot of information. First is the 8-byte NT response ❷, also called the `NTProofStr` in the protocol's documentation. We'll come back to how this value is calculated in a moment. After the NT response are various parameters about the protocol, including the 8-byte client challenge ❸. NTLMv1 already contained the server challenge to prevent replay, but NTLMv2 added the client challenge to make it harder for an attacker to use the `AUTHENTICATE` token to crack the user's password.

CRACKING USER PASSWORDS

While the NTLM authentication protocol does not directly disclose the user's password, the authentication tokens generate values that are causally related to the password. If an attacker can get a user to authenticate to a service they control, they can use the values from the tokens to mount a brute-force attack that retrieves the password's original value, then authenticate as the user.

Brute-forcing can be very time-consuming, especially if the password is long and contains a mix of characters. To speed up password cracking, attackers might use *rainbow tables*, which include many possible precomputed derivations of the authentication token's values for different passwords. This process works best when only the password is unknown; otherwise, an attacker must build a new set of rainbow tables for each unknown value, such as the server challenge. Because the attacker can fix the server challenge when the user

connects but the client challenge is randomly generated, rainbow tables work best with NTLMv1, which doesn't involve a client challenge.

The details of how exactly rainbow tables work are outside the scope of this book, but you can find plenty of resources online if you want to know more about them. Today, rainbow tables have fallen out of favor due to the deprecation of NTLMv1 and the improved performance of commodity computer graphics cards, which has sped up brute-force calculations. If you've captured an NTLMv2 exchange, you can use a tool such as hashcat to brute-force all passwords with fewer than eight characters in less than an hour on a single system. You can also purchase computing resources from a cloud computing platform to mount an attack against more complex passwords.

The `AUTHENTICATE` token copies most of its target information from the `CHALLENGE` message, although it contains a few additional entries. The flags indicate that the message contains a MIC with the `MessageIntegrity` value ④, as you'll soon see. The `SingleHost` flag contains a random ID for the client machine that generated the `AUTHENTICATE` token ⑤. The `ChannelBinding` and `TargetName` ⑥ values are used to prevent credential relaying, an attack we'll also come back to later; in this case, the flags aren't specified. Last is the MIC, a keyed MD5 hash-based message authentication code (HMAC) calculated over the authentication tokens sent and received for the current exchange ⑦. The key for the hash is calculated during the authentication process, and the MIC serves to detect whether the tokens have been tampered with.

The client sends the `AUTHENTICATE` token to the server, which again calls `AcceptSecurityContext`, passing it the token. The LSA runs the calculations needed to verify that the NT response matches the expected value and that the MIC is valid, indicating the tokens haven't been tampered with. If both values match the expected values, the authentication succeeds.

There are several differences between the formatted output generated by NTLMv1 and NTLMv2. First, if NTLMv1 is in use, the NT response in the `AUTHENTICATE` token will be a 24-byte binary value rather than the structured response seen in [Listing 13-4](#). For example, it might look like the following:

NT Response: 96018E031BBF1666211D91304A0939D27EA972776C6C0191

You can also differentiate between NTLMv1 and NTLMv2 Session by looking at the flags and LM hash. If the `ExtendedSessionSecurity` flag is set, you know that NTLMv2 Session is in use; otherwise, the system is using NTLMv1. The LM hash field gets repurposed to contain the client challenge in NTLMv2, which might confuse you, as you might assume an LM hash has been negotiated. You can tell the difference between a hash and a client challenge because the client challenge is only 8 bytes long, as shown here:

LM Response: CB00748C3F04CB5700000000000000000000000000000000

The remaining 16 bytes are padded with zeros.

Requesting a Token Object

Now that the authentication process has completed, the server can request that the LSA generate a Token object for the authenticated user through the `QuerySecurityContextToken` API, as shown in [Listing 13-5](#).

```
PS> Update-LsaServerContext -Server $server -Token $client.Token
PS> if ((Test-LsaContext $client) -and (Test-LsaContext $server)) {
    Use-NtObject($token = Get-LsaAccessToken $server) {
        Get-NtLogonSession -Token $token
    }
}
LogonId          UserName          LogonType SessionId
-----
00000000-0057D74A GRAPHITE\user     Network      0
```

Listing 13-5: Completing the NTLM authentication process

We start by calling `Update-LsaServerContext` again to finalize the authentication process. Once all tokens have been transferred, the client and server contexts are placed into a *done state*, meaning they no longer need any more information to complete the authentication process. You can verify this state using the `Test-LsaContext` command.

With the authentication completed, we can call the `Get-LsaAccessToken` command to return the `Token` object for the user. We display the logon session for the `Token` and verify that it used network authentication.

NETWORK AUTHENTICATION AND LOCAL ADMINISTRATORS

As a result of a network authentication exchange, the LSA generates a `Token` object using the groups and privileges of the local or domain policy. A quirk occurs if the authenticating user is both a local user and a member of the local *Administrators* group and UAC is enabled. In that case, the LSA generates the UAC filtered token for the authentication rather than the full administrator token. This limits a local administrator's ability to access remote services using a local account, as they will no longer be an administrator once authenticated to the remote system, which might prevent them from being able to use the service correctly.

If the Windows system is joined to a domain, domain users won't be limited by this policy if they're added to the local *Administrators* group. For example, Windows adds domain administrators to the local *Administrators* group by default, so they won't be affected by the filtering. You can also disable the filtering by setting a system policy in the registry using the following command; however, as with all system modifications, this can weaken the system's security and you should use it on test systems only:

```
PS> New-ItemProperty -Name "LocalAccountTokenFilterPolicy" -Value 1  
-Force -PropertyType DWORD  
-Path 'HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System'
```

A separate setting, `FilterNetworkAuthenticationTokens`, will always filter network authentication tokens, regardless of where they come from. This setting is disabled by default.

The Cryptographic Derivation Process

The NTLM process never discloses the user's password in plaintext on the network. Even so, NTLM uses the password's value to derive the final NT response and MIC. Let's use PowerShell to walk through this cryptographic derivation process and generate the NT response and MIC. To

perform the derivation, we'll need the user's password, as well as the CHALLENGE and AUTHENTICATE authentication tokens.

We also need a function that calculates the MD5 HMAC for a set of bytes. The MD5 HMAC is a keyed cryptographic hashing algorithm commonly used to sign data so its integrity can be verified. We'll use this function, defined in [Listing 13-6](#), multiple times in the derivation.

```
PS> function Get-Md5Hmac {  
    ❶ Param(  
        $Key,  
        $Data  
    )  
  
    $algo = [System.Security.Cryptography.HMACMD5]::new($Key)  
    if ($Data -is [string]) {  
        $Data = [System.Text.Encoding]::Unicode.GetBytes($Data)  
    }  
    ❷ $algo.ComputeHash($Data)  
}
```

Listing 13-6: Defining the Get-Md5Hmac function

The function is simple: it creates the .NET class `HMACMD5`, passing it a key ❶, then calls `ComputeHash` on the data ❷. If the data is a string, it first converts it to a byte array in Unicode encoding.

The next function we define calculates the *NT one-way function version 2* (*NTOWFv2*), shown in [Listing 13-7](#). This function converts the username, domain, and password into a 16-byte key for further use.

```
PS> function Get-NtOwfV2 {  
    Param(  
        $Password,  
        $UserName,  
        $Domain  
    )  
  
    ❶ $key = Get-MD4Hash -String $Password  
    ❷ Get-Md5Hmac -Key $key -Data ($UserName.ToUpperInvariant() + $Domain)  
}
```

```

❸ PS> $key = Get-NtOwfV2 -Password "pwd" -UserName $authToken.UserName
    -Domain $authToken.Domain
    PS> $key | Out-HexDump
❹ D6 B7 52 89 D4 54 09 71 D9 16 D5 23 CD FB 88 1F

```

Listing 13-7: Defining the NT one-way function

First, note that the system hashes the password using the MD4 algorithm

❶. As mentioned previously, the SAM database stores these MD4 hashes so that the LSA doesn't need to store their plaintext versions.

We supply the MD4 hash of the password as a key to the `Get-Md5Hmac` function, then use this function to hash the uppercase username concatenated to the domain ❷. In this case, these values are `user` and `GRAPHITE`, so we hash the string `USERGRAPHITE`.

To perform this operation, we call the `Get-NtOwfV2` function we just defined with the username and domain from the `AUTHENTICATE` token ❸, which we stored in the `$authToken` variable. The function produces a 16-byte key ❹.

Now that we have a key based on the user's password, we'll use it to calculate the NT response value with the function defined in [Listing 13-8](#).

```

PS> function Get-NtProofStr {
    Param(
        $Key,
        $ChallengeToken,
        $AuthToken
    )

    ❶ $data = $ChallengeToken.ServerChallenge
        $last_index = $AuthToken.NtChallengeResponse.Length - 1
        $data += $AuthToken.NtChallengeResponse[16..$last_index]
    ❷ Get-Md5Hmac -Key $Key -Data $data
    }
    PS> $proof = Get-NtProofStr -Key $key -ChallengeToken $ChallengeToken
        -AuthToken $AuthToken
    PS> $proof | Out-HexDump
    ❸ 53 2B B4 80 4D D9 C9 DF 41 8F 8A 18 D6 7F 55 10

```

Listing 13-8: Calculating the NtProofStr value

We perform the calculation of the NT response using the NTOWFv2 key as well as the CHALLENGE and AUTHENTICATE tokens. First we concatenate the 8-byte server challenge from the CHALLENGE token with the NtChallengeResponse from the AUTHENTICATE token, minus the supplied 16-byte NT response ❶. Then we calculate the NT value using the Get-Md5Hmac function, with NTOWFv2 as the key ❷. The result ❸ should match the NT response value from [Listing 13-4](#) (if you used your actual password rather than the *pwd* placeholder used in the listing).

The server can now verify that the client has access to the correct password for the user by checking whether the two NT response values match. However, we still want to verify that the messages haven't been tampered with in some way, so we need to calculate the MIC. We define the function to do this in [Listing 13-9](#).

```
PS> function Get-Mic {
    Param(
        $Key,
        $Proof,
        $NegToken,
        $ChallengeToken,
        $AuthToken
    )

    ❶ $session_key = Get-Md5Hmac -Key $Key -Data $Proof

    $auth_data = $AuthToken.ToArray()
    ❷ [array]::Clear($auth_data, $AuthToken.MessageIntegrityCodeOffset, 16)
    ❸ $data = $NegToken.ToArray() + $ChallengeToken.ToArray() + $auth_data
    ❹ Get-Md5Hmac -Key $session_key -Data $data
}
PS> $mic = Get-Mic -Key $key -Proof $proof -NegToken $NegToken
    -ChallengeToken $ChallengeToken -AuthToken $AuthToken
PS> $mic | Out-HexDump
❺ F0 E9 5D BE B5 3C 88 5C 06 19 FB 61 C5 AF 59 56
```

Listing 13-9: Calculating the message integrity code

The Get-Mic function takes five parameters: the NTOWFv2 key, the NT response, and all three tokens transmitted back and forth between the client and server. The first task is to use Get-Md5Hmac again to calculate a session key ❶. We use the NTOWFv2 key for this HMAC operation and

supply the NT response as data. Next, we zero the MIC field in the `AUTHENTICATE` token ❷, then concatenate the tokens ❸. We pass the session key and the concatenated tokens to `Get-Md5Hmac` to generate the MIC ❹. The value of the MIC ❺ should match the one generated in [Listing 13-4](#).

Pass-Through Authentication

For the client and server to successfully authenticate using NTLM, both parties must know the user's password (or, more precisely, its NT hash). If you're authenticating to a stand-alone machine, the password must be set in the machine's local SAM database. Configuring this value isn't too difficult in a small network, but on a large network consisting of many machines, doing it manually becomes unmanageable.

On a domain network, the domain controller is responsible for managing the user's NT hash. So how can NTLM function in such an environment? The Netlogon service on the domain controller supports the concept of *pass-through authentication* to facilitate NTLM authentication on other systems in the domain. [Figure 13-2](#) provides an example of the NTLM authentication process in a domain.

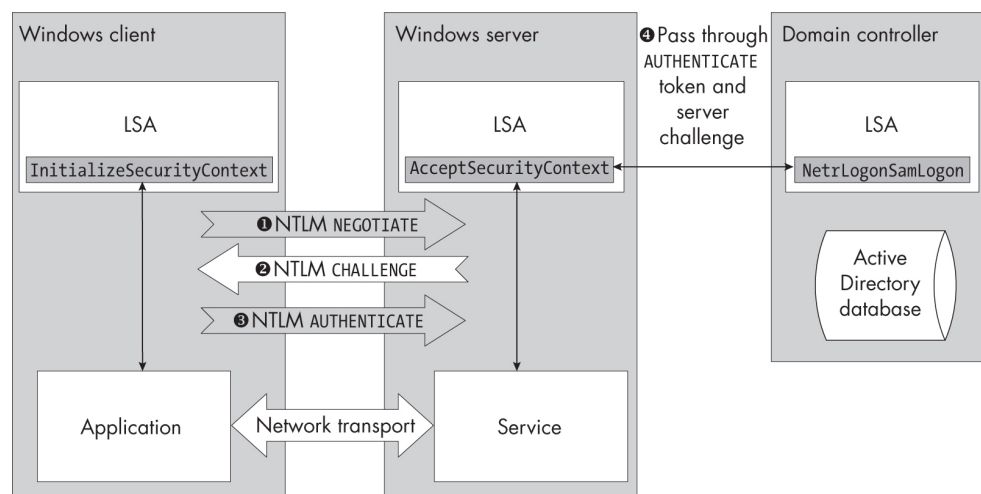


Figure 13-2: An overview of NTLM pass-through authentication

The NTLM authentication process begins normally: the client sends the `NEGOTIATE` token to the server ❶, which generates a challenge and returns it to the client in the `CHALLENGE` token ❷. The client then uses the user's NT hash to generate the `AUTHENTICATE` token and sends it to the server ❸.

At this point, problems arise. The server doesn't have the user's NT hash, so it can't derive necessary cryptographic values such as the NT challenge. Therefore, the server packages up the server challenge and the `AUTHENTICATE` token and sends these to the `NetrLogonSamLogon` API on the domain controller ④. You might recall from [Chapter 12](#) that Windows uses this API for interactive authentication. The API has multiple modes, one of which can verify the NTLM authentication values without needing the user's password.

Note that the domain controller doesn't verify the MIC, as this requires all three authentication tokens. Instead, the server calculates the session key used for verification based on the user's NT hash and NT challenge value and returns it to the requesting server. This allows it to ensure that the authentication hasn't been tampered with.

The Windows server never has access to the user's full password or NT hash, only the session key. This results in the *double hop problem*: the authenticated user can access resources stored locally on the server, but that user cannot be used to access resources on other servers on the domain network.

From a security perspective, this is a good thing, as it prevents a malicious service from repurposing a user's identity. However, it also reduces flexibility, as it means that you can't trivially implement an authenticated proxying service without requiring the user to reauthenticate to each service behind that proxy. Kerberos solves the double hop problem using delegation, as I'll describe in more detail in [Chapter 14](#).

Local Loopback Authentication

In the previous example, I chose to specify a username and domain when getting the outbound authentication credentials handle. While Integrated Windows Authentication doesn't require you to specify either a username or a domain, you need to do so if you want to create a network logon session on the local machine. Let's change the script in [Listing 13-1](#) to build the outbound credentials without a username or domain:

```
PS> $credout = New-LsaCredentialHandle -Package "NTLM" -UseFlag Outbound
```

Now rerun the authentication session. The formatted tokens should look like those in [Listing 13-10](#).

```
<NTLM NEGOTIATE>
Flags: Unicode, Oem, RequestTarget, NTLM, OemDomainSupplied,
OemWorkstationSupplied, AlwaysSign, ExtendedSessionSecurity, Version,
Key128Bit, Key56Bit
❶ Domain: DOMAIN
Workstation: GRAPHITE
Version: 10.0.XXXXX.XX

<NTLM CHALLENGE>
❷ Flags      : Unicode, RequestTarget, NTLM, LocalCall,...
TargetName: DOMAIN
Challenge  : 9900CFB9C182FA39
❸ Reserved  : 5100010000000000
Version    : 10.0.XXXXX.XX
--snip--

<NTLM AUTHENTICATE>
❹ Flags      : Unicode, RequestTarget, NTLM, LocalCall,...
❺ LM Response:
NT Response:
Version      : 10.0.XXXXX.XX
MIC          : 34D1F09E07EF828ABC2780335EE3E452

PS> Get-NtLogonSession -Token $token
LogonId          UserName          LogonType          SessionId
-----
❻ 00000000-000A0908 GRAPHITE\user      Interactive        2

PS> Get-NtTokenId -Authentication
LUID
----
❼ 00000000-000A0908
```

Listing 13-10: The formatted tokens from a local loopback authentication

You might notice that all three authentication tokens have changed. The first change is in the `NEGOTIATE` token, which now contains a domain name and workstation name ❶. The next changes are in the `CHALLENGE` token: a new flag has appeared, `LocalCall` ❷, and a previously zeroed `Reserved` field now has a value ❸. The `LocalCall` flag signifies that the authentication

comes from the local machine, while the `Reserved` field is a unique identifier for the server security context that created the `CHALLENGE` token.

The final changes are in the `AUTHENTICATE` token. While the `LocalCall` flag is still present ❹, both the `LM Response` and `NT Response` fields are completely empty ❺. This clearly signifies that the authentication process has changed. If we check the final `Token` object's logon session, we see that it's an interactive session instead of a network session ❻. The reason for this is that the LSA has returned a copy of the caller's token to the server, as you can see by comparing the logon ID to the authentication ID from the effective token ❼.

Let's take a closer look at the `LocalCall` flag. Its value is based on the domain and workstation names in the `NEGOTIATE` authentication token. If these values refer to the local machine, local loopback authentication is enabled. There are no other unique identifiers in the initial token to key the flag on, and there doesn't need to be an ongoing outbound authentication process for the flag to be selected. Also, the flag is not specified in the `NEGOTIATE` token's flags, so it's not negotiated between the client and server.

At the time of writing, Microsoft does not document the `LocalCall` flag in *MS-NLMP*, presumably because it shouldn't be supported outside of the local machine. However, as you can see, merely providing the right `NEGOTIATE` token causes local loopback authentication to kick in. Documenting this flag would make it easier to diagnose authentication failures that could occur if the flag were present over the network.

Why does the LSA implement local loopback authentication? One reason is that network authentication would cause the user to be reauthenticated, and some local services, such as SMB, allow local interactive users, but not network users, to access file shares. Therefore, this local loopback allows the SMB server to see a local user and grant access.

Alternative Client Credentials

We've seen how to use PowerShell commands to authenticate as the calling user. This is normally the behavior you'll want to implement, as the current user typically aims to access some network resource as themselves. However, the underlying APIs support several mechanisms that allow you to authenticate as a different user over the network. Changing

your user identity is useful because it enables you to access a network resource without reauthenticating interactively.

Using Explicit Credentials

If you know the new user's full credentials, you can specify them when creating the credentials handle for the client authentication context. To do this, call `New-LsaCredentialHandle` and pass it the `UserName`, `Domain`, and `Password` parameters.

However, you probably don't want to leave a user's password in PowerShell's command history. One alternative is to specify the `ReadCredential` parameter, which will read the credentials from the user without storing them in the command history. [Listing 13-11](#) shows an example.

```
PS> $cout = New-LsaCredentialHandle -Package NTLM -UseFlag Outbound
          -ReadCredential
PS> UserName: admin
PS> Domain: GRAPHITE
PS> Password: *****
```

Listing 13-11: Creating a credentials handle with user-specified credentials

You can now pass the credentials handle to `New-LsaClientContext` to create the client context. You don't need to change the server side, which uses the credentials managed by the LSA.

Impersonating a Token

When creating the credentials handle, the LSA usually determines the network credentials to use based on the calling user's identity, which it retrieves from the primary token of the process that calls the SSPI API. However, if you have a different user's token, you can impersonate them while creating the credentials handle to use a different identity. Run the command in [Listing 13-12](#) as an administrator.

```
PS> $credout = Invoke-NtToken -System {
          New-LsaCredentialHandle -Package "NTLM" -UseFlag Outbound
        }
```

Listing 13-12: Creating the credentials handle for the SYSTEM user

In [Listing 13-12](#), we create a credentials handle for the *SYSTEM* user. The *SYSTEM* user doesn't have any explicit password you can use to authenticate using the approach in [Listing 13-11](#); therefore, you must impersonate the token to create a credentials handle for it.

You need to impersonate the token only once, when calling the `NewLsaCredentialHandle` command. All subsequent calls used to create and update the client context don't require you to impersonate the token.

If you have the full credentials, another approach you could use is to create the token with the `NewCredentials` logon type, briefly mentioned in [Chapter 12](#). This will create a token with the same local user identity but replace the network authentication credentials, as illustrated in [Listing 13-13](#).

```
PS> $password = Read-Host -AsSecureString -Prompt "Password"
PS> $new_token = Get-NtToken -Logon -LogonType NewCredentials
               -User "Administrator" -Domain "GRAPHITE" -SecurePassword $password
PS> $credout = Invoke-NtToken $new_token {
               New-LsaCredentialHandle -Package "NTLM" -UseFlag Outbound
               }
```

Listing 13-13: Creating a credentials handle with a NewCredentials token

Here, we create a credentials handle by using the `Get-NtToken` command to generate a `NewCredentials`-type token, then impersonating it when calling `NewLsaCredentialHandle`.

You might be wondering why, if you know the full credentials, you wouldn't just specify them directly when creating the credentials handle. In this example, this would indeed be the simpler solution. However, you sometimes won't have direct control over the creation of the credentials handle. This can happen if the network authentication occurs within another API that uses the caller's identity to access a remote resource. In that case, you can impersonate the `NewCredentials` token while calling the API to use the credentials you specified. Importantly, only the network credentials will change due to impersonation; the local identity will stay the same, so you won't accidentally access local resources with the wrong user account.

Let's finish this chapter by describing a practical attack against the NTLM authentication protocol. This attack allows you to repurpose the credentials of another user without needing to know the user's password.

The NTLM Relay Attack

One thing you might notice about NTLM is that, while the LSA performs the authentication, it's up to the client and server applications to transport the authentication tokens. How does the LSA ensure that it's authenticating to the right computer? It can't do this directly; it needs the help of the client and server applications. This causes a security vulnerability that an actor could exploit with an attack called an *NTLM relay*. In this section, we'll explore this attack and how Microsoft has tried to fix the vulnerability.

Attack Overview

Figure 13-3 shows the basic setup of an NTLM relay attack.

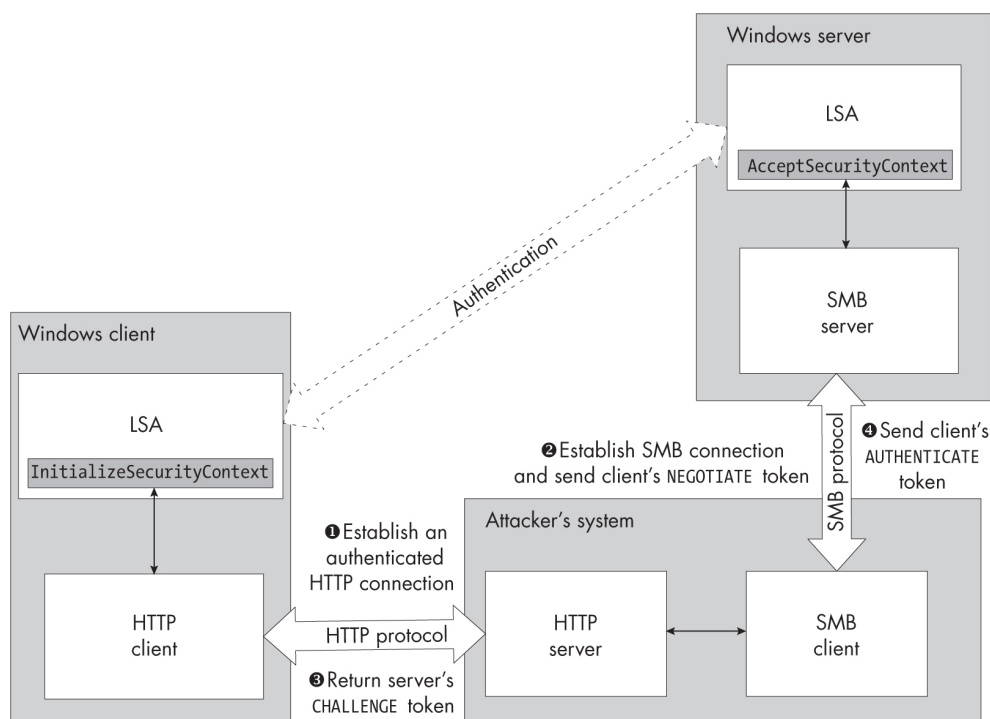


Figure 13-3: An example of an NTLM relay attack

Three systems are involved: a Windows client machine, a Windows server, and the attacker's machine. The attacker's goal is to access the SMB file share on the server. However, they don't have the credentials necessary to successfully perform NTLM authentication. The client, on

the other hand, does have suitable credentials, and because of Interactive Windows Authentication, it will use those credentials without user interaction if asked nicely.

The first step is for the attacker to convince the client machine to connect to the attacker's web server. While the attacker wants to access SMB, the NTLM authentication from the client can be over any protocol that supports authentication, including HTTP. Convincing the client to make a connection could be as simple as adding an image to a web page the client visits that points to the attacker's web server.

The attacker accepts the client's HTTP connection and starts the NTLM authentication process, which results in the client sending a `NEGOTIATE` token to the attacker ❶. Instead of processing the token, the attacker now opens a new connection to the target SMB server and passes along the `NEGOTIATE` token as if they had created it ❷.

The SMB server will respond with a `CHALLENGE` token, and the attacker can forward this to the client to continue the authentication process ❸. The client should respond with an `AUTHENTICATE` token to the attacker's web server, which it can forward to the SMB server ❹. Assuming the server accepts the client's credentials, the attacker has now established an authenticated connection to the SMB server without ever knowing the user's password.

This attack is a serious security issue. Microsoft has tried to implement various fixes, mainly by adding more features to NTLM. However, the problem with these fixes is that they're opt-in, for backward compatibility reasons: NTLM and SMB are such old protocols that certain clients and servers don't support the new features. Still, let's discuss the ways that Windows mitigates the vulnerability.

Active Server Challenges

The simplest way of performing an NTLM relay attack is to authenticate back to the victim's machine. For example, in **Figure 13-3**, the HTTP client and the SMB server could live on the same Windows machine. If the machine is both the client and the server, the authentication credentials will always be valid.

To fix this attack, Windows began maintaining a table of currently active server challenges and refusing to create the `AUTHENTICATE` token if the `CHALLENGE` token included a server challenge issued by the same machine. There is a small chance of a collision occurring between two machines, but with a random 8-byte challenge, this will rarely happen.

Signing and Sealing

Another way of combatting the NTLM relay attack is to make the outer protocol containing the NTLM authentication, such as SMB, rely on the authentication process in some way. This boils down to using the only piece of information the attacker doesn't have: the user's password.

The SSPI APIs and NTLM support the inclusion of a randomly generated session key in the `AUTHENTICATE` token that is encrypted by the user's password. This session key can then be used to generate a MIC, which the documentation refers to as *signing*. The MIC is generated for the outer protocol using the `MakeSignature` SSPI API and verified using the `VerifySignature` API. The key can also be used to encrypt and decrypt arbitrary data using the `EncryptMessage` and `DecryptMessage` APIs, which the documentation refers to as *sealing*. Because the attacker can't decrypt the session key without knowing the password, they can't generate valid signed or encrypted data to communicate with the relayed server.

To request a session key, you specify the `Confidentiality OR Integrity` flag when creating the client or server context by using the `RequestAttribute` parameter. For example, when calling `New-LsaClientContext`, you can specify the following command:

```
PS> $client = New-LsaClientContext -CredHandle $credout -RequestAttribute Integrity
```

Listing 13-14 shows the client's `AUTHENTICATE` token if we specify the `Integrity` request attribute flag when creating the client and server contexts.

```
<NTLM AUTHENTICATE>
❶ Flags      : Unicode, RequestTarget, Signing, NTLM, AlwaysSign,
ExtendedSessionSecurity, TargetInfo, Version,
Key128Bit, KeyExchange, Key56Bit
```

```
--snip--
</NTLMv2 Challenge Response>
❷ Session Key: 5B13E92C08E140D37E156D2FE4B0EAB9
Version      : 10.0.18362.15
MIC          : 5F5E9B1F1556ADA1C07E83A715A7809F
```

Listing 13-14: Checking the AUTHENTICATE token for the session key

As the output shows, this changes the NTLM process in two important ways. First, the NTLM `KeyExchange` flag has been added ❶. This flag indicates that the client has generated a session key. The flags also now include `Signing`, which indicates to the server that the client wants to allow the signing of content based on the session key. If the `Confidentiality` request attribute flag is used, two `AUTHENTICATE` flags are set, `Signing` and `Sealing`.

If either flag is set, the NTLMv2 challenge contains an encrypted session key that the client generated ❷. This is the base key used for all further cryptographic operations. The key is encrypted using the RC4 encryption algorithm and a key derived from the user's hash and the NT response.

If you verify the MIC after enabling signing or sealing, you'll notice that the value generated no longer matches the one in the `AUTHENTICATE` token. This is because if the encrypted session key is available, it's used instead of the base session key. You can fix this behavior by modifying the `Get-Mic` function shown in [Listing 13-9](#), adding the bold portion in [Listing 13-15](#).

```
$session_key = Get-Md5Hmac -Key $Key -Data $Proof
if ($authToken.EncryptedSessionKey.Count -gt 0) {
    $session_key = Unprotect-RC4 -Key $session_key
    -Data $AuthToken.EncryptedSessionKey
}
```

Listing 13-15: Modifying the Get-Mic function to decrypt the session key for the MIC calculation

The `MakeSignature` and `VerifySignature` APIs are exposed through the `Get-LsaContextSignature` and `Test-LsaContextSignature` commands, while the `EncryptMessage` and `DecryptMessage` APIs are exposed through the `Protect-LsaContextMessage` and `Unprotect-LsaContextMessage` commands. We'll cover the

use of these encryption commands in the worked example at the end of this chapter; for now, [Listing 13-16](#) shows a simple use of the signature commands.

```
❶ PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext $server $client
PS> Update-LsaClientContext $client $server
PS> Update-LsaServerContext $server $client
PS> $msg = $(0, 1, 2, 3)
❷ PS> $sig = Get-LsaContextSignature -Context $client -Message $msg
PS> $sig | Out-HexDump
01 00 00 00 A7 6F 57 90 8B 90 54 2B 00 00 00 00

❸ PS> Test-LsaContextSignature -Context $server -Message $msg -Signature $sig
True

❹ PS> Test-LsaContextSignature -Context $server -Message $msg -Signature $sig
False
```

Listing 13-16: Generating and verifying a message signature

We start by completing the client-to-server authentication process to set up integrity support ❶. We then generate a signature for a simple 4-byte message using the client authentication context ❷. This process assumes that the data is being sent to the server for verification; we can reverse it by specifying a different authentication context. We display the generated signature value as hex.

We then verify the signature with the server authentication context using the `Test-LsaContextSignature` command ❸. The command returns a Boolean value indicating whether the signature is valid. For this call, the verification returns `True`. However, if we check the signature a second time ❹, we now get `False`, indicating that it is no longer valid. Why is that?

The client and server authentication contexts maintain a *sequence number*, which starts at 0 and increments for every signature or encryption operation. This sequence number is automatically included when generating or verifying a signature, and the server can use it to check whether an old signature has been replayed (for example, if an attacker is trying to send the same network data twice).

In the example in [Listing 13-16](#), we generated the client's signature with a sequence number of 0. In the first verification, the server's authentication context also has an initial value of 0, so the verification succeeds.

However, after the verification completes, the server's sequence number is incremented to 1. So, when we try to verify the same signature again, the sequence numbers no longer match, and the verification fails.

The RC4 encryption algorithm used for signing and sealing has numerous weaknesses, which are outside the scope of this book. However, it offers some level of mitigation against NTLM relay attacks and provides basic integrity and confidentiality protections to the outer network protocol if no other key exchange mechanism is in place.

SMB supports signing and encryption derived from the authentication process. However, because of the weakness of RC4, SMB doesn't use the `MakeSignature` or `EncryptMessage` APIs; instead, it extracts the decrypted session key using the `QueryContextAttribute` SSPI API and uses its own encryption and integrity-checking algorithms. You can query for the session key by accessing the `SessionKey` property on the client or server authentication context, as shown in [Listing 13-17](#).

```
PS> $server.SessionKey | Out-HexDump
F3 FA 3A E0 8D F7 EE 34 75 C5 00 9F BF 77 0E E1
PS> $client.SessionKey | Out-HexDump
F3 FA 3A E0 8D F7 EE 34 75 C5 00 9F BF 77 0E E1
```

Listing 13-17: Extracting the session keys for the authentication context

Target Names

Another technique for blocking NTLM relay attacks is to add an identifier to the `AUTHENTICATE` token that indicates the name of the target the NTLM authentication is for. Because the `AUTHENTICATE` token is protected by the MIC, which is derived from the user's password, the target name is hard to tamper with.

In our NTLM relay example, if the client enabled target names, it might set the target name to `HTTP/attacker.domain.local`, where `HTTP` represents the type of service requested and `attacker.domain.local` is the address to which it's authenticating. The attacker could pass the `AUTHENTICATE`

token to the SMB server, but because the server runs a different service, *CIFS*, and sits on a different network address, *filesrvr.domain.local*, the names will not match and authentication will fail.

To specify a target name, set the `Target` parameter when creating the client authentication context:

```
PS> $client = New-LsaClientContext -CredHandle $credout -Target "HTTP/localhost"
```

Note that the target name can be completely arbitrary, but the service type or network address can't be. For example, the name *BLAH* wouldn't be rejected, but the name *BLAH/microsoft.com* would be (unless you happened to be running a server on *microsoft.com*). The name format follows that of the service principal name (SPN) used in Kerberos authentication. We'll describe how Kerberos uses SPNs in the next chapter.

When you run the NTLM authentication, you should now see the target name in the NTLMv2 challenge response block:

```
TargetName - HTTP/localhost
```

You can extract the target name from the server authentication content with the `ClientTargetName` property:

```
PS> $server.ClientTargetName  
HTTP/localhost
```

The problem with the target name protection is that it must be enabled to be effective. By default, clients won't set it, and the SMB server does not require it to be specified. Also, an attacker can spoof the name, as it's typically based on some network address. For example, the attacker might be able to poison the client's DNS cache or use other local network attacks to hijack the server's IP address.

Channel Binding

The final protection against NTLM relay we'll discuss is *channel binding*, which Microsoft also refers to as *Extended Protection for Authentication*

(EPA). The purpose of channel binding is to add an additional value to the NTLMv2 `AUTHENTICATE` token that the MIC will protect from tampering.

Instead of using an arbitrary name, channel binding allows the client and server to specify a binary token related to some property of the outer network protocol. One common use of channel binding is in *Transport Layer Security (TLS)*, a generic network protocol that encrypts and verifies another streaming protocol. This prevents the encrypted protocol's contents from being disclosed to anyone inspecting network traffic and enables tampering detection. It's used, for example, to secure HTTP as HTTPS.

In a TLS communication, the client and server could specify the TLS server's X.509 certificate as the channel binding token. The TLS protocol first verifies the certificate and ensures that the connection is really being made to the destination server. Then it binds the NTLM authentication to that channel. This prevents attackers from hijacking authentication by injecting data into the TLS channel. If the attacker instead redirects a TLS connection to their own server, the certificate will be different, and will use a different channel binding value.

To enable channel binding, specify the `ChannelBinding` parameter in the client and server authentication contexts:

```
PS> $client = New-LsaClientContext -CredHandle $credout -ChannelBinding @(1, 2, 3)
PS> $server = New-LsaServerContext -CredHandle $credin -ChannelBinding @(1, 2, 3)
```

If you now run the NTLM authentication process, you'll find that the channel binding value, which used to be all zeros, now has a value similar to the following:

```
ChannelBinding - BAD4B8274DC394EDC375CA8ABF2D2AEE
```

The `ChannelBinding` value is an MD5 hash of a `SEC_CHANNEL_BINDINGS` structure, which includes the channel binding data specified to the authentication context. The value itself should always be the same for every authentication with the same data. For the implementation used in the PowerShell module, you can use the function in [Listing 13-18](#) to calculate the hash.

```
PS> function Get-BindingHash {  
    Param(  
        [byte[]]$ChannelBinding  
    )  
    $stm = [System.IO.MemoryStream]::new()  
    $writer = [System.IO.BinaryWriter]::new($stm)  
    $writer.Write(0) # dwInitiatorAddrType  
    $writer.Write(0) # cbInitiatorLength  
    $writer.Write(0) # dwAcceptorAddrType  
    $writer.Write(0) # cbAcceptorLength  
    $writer.Write($ChannelBinding.Count) # cbApplicationDataLength  
    $writer.Write($ChannelBinding) # Application Data  
    [System.Security.Cryptography.MD5Cng]::new().ComputeHash($stm.ToArray())  
}  
PS> Get-BindingHash -ChannelBinding @(1, 2, 3) | Out-HexDump  
BA D4 B8 27 4D C3 94 ED C3 75 CA 8A BF 2D 2A EE
```

Listing 13-18: Calculating the channel binding hash

As with target names, systems must opt in to this feature. If the server does not specify a channel binding token, the channel binding hash in the `AUTHENTICATE` token won't be checked. Only when the server specifies a channel binding token that doesn't match will the authentication process fail.

Worked Example

Let's finish with a worked example using the commands you've learned about in this chapter. In this example, we'll develop a simple network protocol that uses NTLM and the authentication context mechanisms to authenticate a user over a network, providing encryption and integrity verification. As this example will be quite complex, I'll break it into sections.

Overview

The .NET framework already comes with the `NegotiateStream` class, which uses the SSPI to authenticate and encrypt network communications. Nevertheless, you'll find it instructive to build a similar mechanism yourself. The network protocol we'll develop won't be robust or even secure; it will merely demonstrate a practical use of the commands described in this chapter.

NTLM's security properties (and its encryption and integrity verification mechanisms) are very weak by modern standards, so if you want a robust encrypted network protocol, use TLS instead. TLS is available through the `SslStream` class in .NET.

Figure 13-4 shows a basic overview of the protocol we'll build.

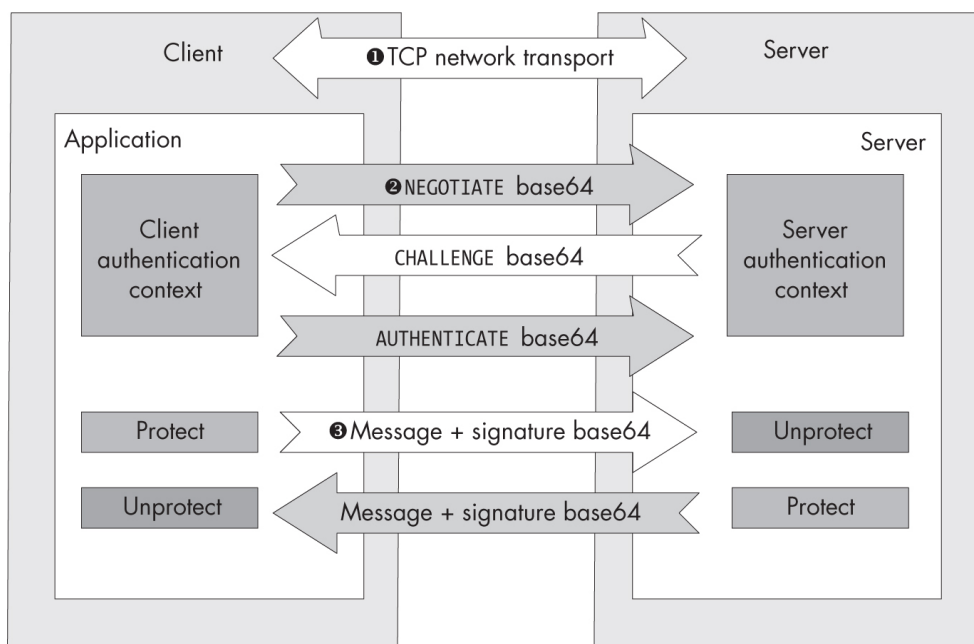


Figure 13-4: An overview of the network protocol

We'll use TCP to facilitate communications between the client and the server **1**. TCP is a reliable protocol built into almost every computing device on the planet—but because it's a streaming protocol, there are no breaks between messages you send or receive. We need a way of breaking up the stream so that the client and server know when they've read a single message. For simplicity, we'll send data as ASCII text, with a new-line character at the end to indicate the end of a message.

Once we've established the TCP connection, we'll perform an NTLM authentication **2**. As the authentication tokens for NTLM are binary, we'll encode them using the base64 algorithm, which converts binary data into a text string made of 64 ASCII characters.

We can then send messages back and forth between the client and the server **3**. We'll encrypt and decrypt the data using the `Protect-`

`LsaContextMessage` and `Unprotect-LsaContextMessage` PowerShell commands. As

the encryption process generates encrypted messages and a separate signature, we'll send them as two separate base64 text lines.

The Code Module

The client and server will perform many of the same tasks, such as sending and receiving messages, so it makes sense to put that code into a separate module that both sides can easily reference. Create a directory for the example code and copy [Listing 13-19](#) into its own file with the name *network_protocol_common.psm1*, as both the server and client implementations will need to access it.

```
Import-Module NtObjectManager
function Get-SocketClient {
    param(
        [Parameter(Mandatory)]
        $Socket
    )

    $Socket.Client.NoDelay = $true
    $stream = $Socket.GetStream()
    $reader = [System.IO.StreamReader]::new($stream)
    $writer = [System.IO.StreamWriter]::new($stream)
    $writer.AutoFlush = $true
    return @{
        Reader = $reader
        Writer = $writer
    }
}

function Send-Message {
    param(
        [Parameter(Mandatory)]
        $Client,
        [Parameter(Mandatory)]
        $Message
    )

    Write-Verbose "Sending Message"
    Format-HexDump -Byte $Message -ShowAll | Write-Verbose
    $text = [System.Convert]::ToBase64String($Message)
    $Client.Writer.WriteLine($text)
}
```

```
function Receive-Message {
    param(
        [Parameter(Mandatory)]
        $Client
    )

    $text = $Client.Reader.ReadLine()
    $ba = [System.Convert]::FromBase64String($text)
    Write-Verbose "Received Message"
    Format-HexDump -Byte $ba -ShowAll | Write-Verbose

    Write-Output -NoEnumerate $ba
}

function Send-TextMessage {
    param(
        [Parameter(Mandatory)]
        $Client,
        [Parameter(Mandatory)]
        $Message,
        [Parameter(Mandatory)]
        $Context
    )

    $bytes = [System.Text.Encoding]::UTF8.GetBytes($Message)
    $enc = Protect-LsaContextMessage -Context $Context -Message $bytes
    Send-Message -Client $Client -Message $enc.Message
    Send-Message -Client $Client -Message $enc.Signature
}

function Receive-TextMessage {
    param(
        [Parameter(Mandatory)]
        $Client,
        [Parameter(Mandatory)]
        $Context
    )

    $msg = Receive-Message -Client $Client
    if ($msg.Length -eq 0) {
        return ""
    }

    $sig = Receive-Message -Client $Client
    if ($sig.Length -eq 0) {
        return ""
    }
}
```

```
}  
  
$dec = Unprotect-LsaContextMessage -Context $Context -Message $msg -Signature  
[System.Text.Encoding]::UTF8.GetString($dec)  
}  
  
Export-ModuleMember -Function 'Get-SocketClient', 'Send-Message',  
'Receive-Message', 'Send-TextMessage', 'Receive-TextMessage'
```

Listing 13-19: The shared module code for the protocol

The module code contains five functions. The first function, `Get-SocketClient`, accepts a connected TCP socket and creates a `StreamReader` and a `StreamWriter` class. These classes allow you to read and write text lines to a binary stream, in this case over the network. We also set the socket's `NoDelay` property, which disables something called the Nagle algorithm. The details of the algorithm are outside the scope of this book, but it ensures that the data written to the socket is sent to the network immediately, rather than being buffered.

The next two functions, `Send-Message` and `Receive-Message`, send and receive a binary message over the TCP socket. To send a message, we first convert the binary data to a base64 string, then write it to the writer object. For the receiving function we do the reverse operation, reading a line from the TCP socket and converting it back to binary data from base64. Note that we're printing the messages we're sending and receiving using the `Write-Verbose` PowerShell command. By default, PowerShell won't show this verbose output; I'll show you how to enable that later.

The final two functions, `Send-TextMessage` and `Receive-TextMessage`, send and receive encrypted text messages. To send an encrypted message, we convert the message into binary data using the UTF8 text encoding, which allows us to use any Unicode character in our string. We then encrypt the binary data using the `Protect-LsaContextMessage` command. We must send the encrypted data and signature as separate lines, using our existing `Send-Message` command. Again, to receive data, we perform the inverse of the sending operation.

The Server Implementation

We'll start by implementing the server, as without a server it will be hard to test any client code. [Listing 13-20](#) contains the server implementation.

```

❶ param(
    [switch]$Global,
    [int]$Port = 6543
)

❷ Import-Module "$PSScriptRoot\network_protocol_common.psm1"
$socket = $null
$listener = $null
$context = $null
$credin = $null

try {
    ❸ $Address = if ($Global) {
        [ipaddress]::Any
    } else {
        [ipaddress]::Loopback
    }

    ❹ $listener = [System.Net.Sockets.TcpListener]::new($Address, $port)
    $listener.Start()
    $socket = $listener.AcceptTcpClient()
    $client = Get-SocketClient -Socket $socket
    Write-Host "Connection received from $($socket.Client.RemoteEndPoint)"

    ❺ $credin = New-LsaCredentialHandle -Package "NTLM" -UseFlag Inbound
    $context = New-LsaServerContext -CredHandle $credin
    -RequestAttribute Confidentiality

    ❻ $neg_token = Receive-Message -Client $client
    Update-LsaServerContext -Server $context -Token $neg_token
    Send-Message -Client $client -Message $context.Token.ToArray()
    $auth_token = Receive-Message -Client $client
    Update-LsaServerContext -Server $context -Token $auth_token

    if (!(Test-LsaContext -Context $context)) {
        throw "Authentication didn't complete as expected."
    }

    ❼ $target = "BOOK/$($socket.Client.LocalEndPoint.Address)"
    if ($context.ClientTargetName -ne $target) {

```

```

        throw "Incorrect target name specified: $($context.ClientTargetName)."
    }

    $user = Use-NtObject($token = Get-LsaAccessToken -Server $context) {
        $token.User
    }
    Write-Host "User $user has authenticated."
    ❶ Send-TextMessage -Client $client -Message "OK" -Context $context

    ❷ $msg = Receive-TextMessage -Client $client -Context $context
    while($msg -ne "") {
        Write-Host "> $msg"
        $reply = "User {0} said: {1}" -f $user, $msg.ToUpper()
        Send-TextMessage -Client $client -Message $reply -Context $context
        $msg = Receive-TextMessage -Client $client -Context $context
    }
} catch {
    Write-Error $_
} finally {
    if ($null -ne $socket) {
        $socket.Close()
    }
    if ($null -ne $listener) {
        $listener.Stop()
    }
    if ($null -ne $context) {
        $context.Dispose()
    }
    if ($null -ne $credin) {
        $credin.Dispose()
    }
}
}

```

Listing 13-20: A simple server implementation

Copy this code into its own script file in the same directory as the module file in [Listing 13-19](#), and save it as *network_protocol_server.ps1*.

We start by defining some parameters ❶. If you use the code as a script, you can make it act like a function by having it accept parameters on the command line. This makes it easy to change the script's behavior. In this case, we define a `Global` parameter, which will change what network interfaces we bind the TCP server to, and a `Port` parameter, which is the TCP port number.

Next, we import the common module ❷. This ensures that the functions defined in [Listing 13-19](#) are available for the server to use. Then we set up the bind address ❸. If `Global` is set, then we bind to `Any`, which represents all network interfaces; if not, we bind only to the loopback address, which is accessible only locally.

NOTE

It's a common practice to bind to only the loopback address when testing server code. This ensures that other computers on the network can't connect to your server and potentially abuse its functionality. Only bind to all network interfaces when you're confident that any code you've written is secure, or when on a network with no other participants.

Once we've determined the address, we create an instance of the `TcpListener` class and bind to the address and TCP port ❹. We call `Start` to begin listening for new connections, and we wait for a connection by calling `AcceptTcpClient`. At this point, without a client, the script will stop here. When a connection is made, we'll receive a connected socket object that we can convert to the client using the `Get-SocketClient` command. We then print out the connected client address.

We can now set up a new server authentication context for NTLM ❺, specifying the `Confidentiality` request attribute to grant us the ability to encrypt and decrypt messages. We then negotiate the authentication with the client ❻. If the authentication fails or we haven't completed it after receiving the `AUTHENTICATE` token, we throw an error to stop the server script.

We also check that the client provides a suitable target name during the authentication ❼. It should be of the format `BOOK/<ADDRESS>`, where `<ADDRESS>` is the IP address of the server. If the target name doesn't match, we'll also throw a fatal error. To confirm the identity of the authenticated user, we query the `Token` object from the context and print the user's name. To inform the client that the authentication succeeded, we send a confirmation message ❽. We encrypt this message, to ensure the session keys match.

Finally, we can start receiving text messages from the client ❾. We read a text message, which we saw earlier will be decrypted and verified based

on the negotiated authentication context. To prove it was received correctly, we write the message to the console. We then return the message to the client, appending the username to the message and uppercasing the text just for good measure.

If we receive an empty message, we treat this as the signal to close down the server; we'll only accept the one connection. We make sure to clean up our resources, such as the TCP server, before leaving the script. Let's now look at the client implementation.

The Client Implementation

For the most part, the client implements the reverse operations of the server. [Listing 13-21](#) shows its code. Copy this into its own script file in the same directory as the module file from [Listing 13-19](#), with the name *network_protocol_client.ps1*.

```
❶ param(
    [ipaddress]$Address = [ipaddress]::Loopback,
    [int]$Port = 6543
)

Import-Module "$PSScriptRoot\network_protocol_common.psm1"

$socket = $null
$context = $null
$credout = $null

try {
    ❷ $socket = [System.Net.Sockets.TcpClient]::new()
    $socket.Connect($Address, $port)
    $client = Get-SocketClient -Socket $socket
    Write-Host "Connected to server $($socket.Client.RemoteEndPoint)"

    ❸ $credout = New-LsaCredentialHandle -Package "NTLM" -UseFlag Outbound
    $context = New-LsaClientContext -CredHandle $credout
    -RequestAttribute Confidentiality -Target "BOOK/$Address"
    Send-Message -Client $client -Message $context.Token.ToArray()
    $chal_token = Receive-Message -Client $client
    Update-LsaClientContext -Client $context -Token $chal_token
    Send-Message -Client $client -Message $context.Token.ToArray()

    if (!(Test-LsaContext -Context $context)) {
```

```

        throw "Authentication didn't complete as expected."
    }

    ❶ $ok_msg = Receive-TextMessage -Client $client -Context $context
    if ($ok_msg -ne "OK") {
        throw "Failed to authenticate."
    }

    ❷ $msg = Read-Host -Prompt "MSG"
    while($msg -ne "") {
        Send-TextMessage -Client $client -Context $context -Message $msg
        $recv_msg = Receive-TextMessage -Client $client -Context $context
        Write-Host "> $recv_msg"
        $msg = Read-Host -Prompt "MSG"
    }

} catch {
    Write-Error $_
} finally {
    if ($null -ne $socket) {
        $socket.Close()
    }
    if ($null -ne $context) {
        $context.Dispose()
    }
    if ($null -ne $credout) {
        $credout.Dispose()
    }
}
}

```

Listing 13-21: The client implementation

Again, we start by defining some parameters ❶. In this case, we want to specify an IP address to connect to and its TCP port. By default, the client will connect to the loopback address on TCP port 6543. Next, we need to create the TCP socket ❷. Because it's a client, we can directly create a `TcpClient` object to connect to the address and port. We can then wrap the socket with the stream readers and writers, like in the server implementation.

We create a client authentication context so that we can authenticate to the server ❸. We'll use the current user's credentials for this purpose, but you can change this behavior if necessary. We also specify the target name so it matches the server's; if we don't do this, the server will discon-

nect us. We verify that we can read the ok message sent from the server ④. If we don't receive anything or the message does not match our expectations, it's clear the authentication failed.

NOTE

You generally shouldn't return detailed error information to a client in a network protocol. Sending a simple OK message, or nothing at all, may not help diagnose problems, but it prevents an attacker from finding out why the authentication failed. For example, if we sent the client the message BADPASSWORD if the password were wrong or BADUSER for an unknown user, an attacker could differentiate the two cases and try to brute-force a password for a valid user or enumerate valid usernames.

If the authentication completed, we should now have a valid connection, so we can start sending messages. We read a text line from the console ⑤ and send it to the server. We then wait for a reply and print it to the console. If we enter an empty line, the loop should exit, and the TCP socket should close. This should cause the server to receive an empty message, at which point the server can exit as well.

The NTLM Authentication Test

Let's test the client and server we've just written. To do so, you'll need two PowerShell consoles. In the first console, run the server script with the following command:

```
PS> .\network_protocol_server.ps1
```

Then, in the second console, run the client. When you see the msg prompt, enter a message, such as `Hello`, to send to the server. The output in the client should resemble the following:

```
PS> .\network_protocol_client.ps1
Connected to server 127.0.0.1:6543
MSG: Hello
> User GRAPHITE\user said: HELLO
MSG:
```

In the server console, the output should show the following:

```
Connection received from 127.0.0.1:60830
User GRAPHITE\user has authenticated.
> Hello
```

Now, if you press ENTER again in the client without typing a message, both the client and the server should exit without any errors.

You can play with the scripts to make them do different things. For example, if you want to use a different TCP port, you can specify the `Port` parameter to the scripts. The following shows how to set the port to 11111 for the server; the change would be the same for the client:

```
PS> .\network_protocol_server.ps1 -Port 11111
```

As a final note, let's revisit the use of the `Write-Verbose` command in the common module code. As you may have noticed when using the client and the server, the verbose output isn't printed to the console. If you want to see the output, you can enable this by changing the value of the `$VerbosePreference` global variable. This variable normally has the value of `SilentlyContinue`, which ignores verbose output. If you change it to `Continue`, the verbose output will appear. [Listing 13-22](#) changes this value before connecting the client.

```
PS> $VerbosePreference = "Continue"
PS> .\network_protocol_client.ps1
VERBOSE: Importing function 'Get-SocketClient'.
VERBOSE: Importing function 'Receive-Message'.
VERBOSE: Importing function 'Receive-TextMessage'.
VERBOSE: Importing function 'Send-Message'.
VERBOSE: Importing function 'Send-TextMessage'.
Connected to server 127.0.0.1:6543
VERBOSE: Sending Message
VERBOSE:      00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F - 0123456789AB
-----
00000000: 4E 54 4C 4D 53 53 50 00 01 00 00 00 B7 B2 08 E2 - NTLMSSP.....
00000010: 09 00 09 00 2D 00 00 00 05 00 05 00 28 00 00 00 - .....(....
--snip--
```

Listing 13-22: Enabling verbose output for the client

You can observe that we now see part of the first NTLM authentication token being sent to the server. When you send messages back and forth between the client and server, you can verify that the data is encrypted by looking at the hex output.

This worked example was quite lengthy, but it should have given you a better idea of how network authentication can work in a real network scenario.

Wrapping Up

This chapter described the NTLM authentication protocol and provided scripts to demonstrate its authentication process. We looked at negotiating authentication tokens, and using the `Format-LsaAuthToken` PowerShell command to display the protocol state.

I also showed you how to derive some of the cryptographic values generated by the NTLM protocol using PowerShell. This included the final NT response value, which proves the knowledge of the user's password, and the message integrity code, which protects the NTLM authentication tokens from tampering.

To describe the risks associated with NTLM authentication, we covered NTLM relay attacks and a few ways in which Windows tries to combat them, such as active server challenge records and channel binding. We also covered using the authentication context to generate signatures and encrypt messages.

Now that you better understand network authentication and the APIs used to generate authentication tokens, the next chapter focuses on the more complicated Kerberos authentication protocol.